第5章 数组和广义表

5.1 数组的类型定义

5.2 数组的顺序表示和实现

5.3 矩阵的压缩存储

5.4 广义表的类型定义

5.5 广义表的存储结构

本章重点难点

- **重点**: 数组的存储表示方法；特殊矩阵压缩存储方法；稀疏矩阵的压缩存储方法。

- **难点**: 稀疏矩阵的压缩存储表示和实现算法。

第5章 数组和广义表

- 5.1 数组的类型定义
- 5.2 数组的顺序表示和实现
- 5.3 矩阵的压缩存储
- 5.4 广义表的类型定义
- 5.5 广义表的存储结构

5.1 数组的定义

□ 数组的定义

数组是由下标 (index) 和值(value)组成的有序对(index，value)的集合。

也可以定义为是由相同类型的数据元素组成有限序列。

□ 数组的特点

数组在内存中是采用一组连续的地址空间存储的，正是由于此种原因，才可以实现下标运算。

所有数组都是一个一维向量。

第5章 数组和广义表

- 5.1 数组的类型定义
- 5.2 数组的顺序表示和实现
- 5.3 矩阵的压缩存储
- 5.4 广义表的类型定义
- 5.5 广义表的存储结构

5.2 数组的顺序表示和实现

□ 数组元素的地址关系

数组在内存中主要采用两种存储方式：

(1)以行序为主的存储方式;

(2)以列序为主的存储方式。

不同的存储方式有不同元素地址计算方法。

5.2 数组的顺序表示和实现

例 分别以行序为主和以列序为主求二维数组a[3][4]中元素a[1][2]地址，首地址用loc(a[0][0])表示，每个元素占用L个内存单位。

a00	a01	a02	a03
a10	a11	a12	a13
a20	a21	a22	a23

以行序为主：

$$loc(a[1][2])=loc(a[0][0])+[(1\times 4)+2]\times L$$

以列序为主：

$$loc(a[1][2])=loc(a[0][0])+[(2\times 3)+1]\times L$$

5.2 数组的顺序表示和实现

□ 数组元素的地址关系（行序为主）

设每个元素所占空间为L,A[0][0]的起始地址记为LOC[0,0]。

二维数组$A[b_1][b_2]$中元素A_{ij}的起始地址为：

$$LOC[i,j]=LOC[0,0]+(b_2\times i+j)\times L$$

三维数组$A[b_1][b_2][b_3]$中数据元素A[i][j][k]的起始地址为：

$$LOC[i,j,k]=LOC[0,0,0]+(b_2\times b_3\times i+b_3\times j+k)\times L$$

第5章 数组和广义表

5.1 数组的类型定义

5.2 数组的顺序表示和实现

5.3 矩阵的压缩存储

5.4 广义表的类型定义

5.5 广义表的存储结构

5.3 矩阵的压缩存储

5.3.1 特殊矩阵

5.3.2 稀疏矩阵

5.3.3 矩阵的压缩存储

5.3.1 特殊矩阵

□ 下(上)三角矩阵

对角线以上(下)元素都为0的矩阵称为下(上)三角矩阵与对称矩阵A

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 2 & 0 & 0 \\ 4 & 5 & 3 & 0 \\ 3 & 6 & 7 & 8 \end{pmatrix} \qquad \begin{pmatrix} 1 & 5 & 6 & 3 \\ 0 & 2 & 1 & 3 \\ 0 & 0 & 0 & 2 \\ 0 & 0 & 0 & 8 \end{pmatrix}$$

下三角矩阵

上三角矩阵

5.3.1 特殊矩阵

- 下(上)三角矩阵的存储方式

下(上)三角矩阵对角线以上(下)元素都为零，根据这个特点可以定义一个长度为n*(n+1)/2的一维数组来存储。

- 下(上)三角矩阵压缩存储时地址对应关系

设下三角矩阵为a[n][n]，用一维数组sa[n*(n+1)/2]存储，则矩阵元素a[i][j]在数组sa中的位置为：

当i>=j时

a[i][j]对应存储在sa[i(i-1)/2+j-1]

5.3.1 特殊矩阵

对称矩阵

$$\begin{bmatrix} 0 & 3 & 0 & 15 \\ 3 & 0 & 4 & 0 \\ 0 & 4 & 0 & 6 \\ 15 & 0 & 6 & 0 \end{bmatrix}$$

- 存储其下三角的值，对称关系映射
- 存储于一维数组sa[0..n(n+1)/2-1]
  - sa[k]和矩阵元$a_{i,j}$之间存在着一一对应的关系：

k	0	1	2	3	…	n(n-1)/2	…	n(n+1)/2-1
sa[k]	a_{00}	a_{10}	a_{11}	a_{20}	…	$a_{n-1,0}$	…	$a_{n-1,n-1}$

$$k = \begin{cases} \dfrac{j(j+1)}{2} + i, 当 i < j \\ \dfrac{i(i+1)}{2} + j, 当 i \geq j \end{cases}$$

5.3.2 稀疏矩阵

◻ 何谓稀疏矩阵

假设 m 行 n 列的矩阵含 t 个非零元素，

称: $\delta = \dfrac{t}{m \times n}$ 为稀疏因子。

通常认为 $\delta \le 0.05$ 的矩阵为稀疏矩阵。

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

矩阵M

5.3.2 稀疏矩阵

□ 稀疏矩阵的三元组顺序表存储

根据稀疏矩阵大部分元素的值都为零的特点，可以只存储稀疏矩阵的非零元素，三元组分别记录非零元素的行，列位置和元素值。

例 求矩阵M的三元组表示。

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

矩阵M

M=((1，2，1)，(2，4，2)，(3，1，1))

5.3.3 矩阵的压缩存储

◻ 三元组顺序表的实现

```
#define  MAXSIZE  12500
typedef struct {
    int  i, j;            //该非零元的行下标和列下标
    ElemType  e;          // 该非零元的值
} Triple;                 // 三元组类型

typedef struct{
     Triple  data[MAXSIZE + 1];    //data[0]未用
      int     mu, nu, tu;
} TSMatrix;                        // 稀疏矩阵类型

TSMatrix M;
```

5.3.3 矩阵的压缩存储

三元组顺序表转值的实现

- 示例分例

(1)从矩阵到转置矩阵

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 5 \\ 0 & 7 & 0 & 0 & 0 \\ 3 & 0 & 0 & 2 & 0 \end{pmatrix} \Rightarrow \begin{pmatrix} 0 & 0 & 3 \\ 1 & 7 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 2 \\ 5 & 0 & 0 \end{pmatrix}$$

矩阵M　　转值矩阵T

5.3.3 矩阵的压缩存储

三元组顺序表转值的实现

- 示例分例

(2)三元组的转值

1	2	1
1	5	5
2	2	7
3	1	3
3	4	2

→

2	1	1
5	1	5
2	2	7
1	3	3
4	3	2

1	2	1
1	5	5
2	2	7
3	1	3
3	4	2

→

1	3	3
2	1	1
2	2	7
4	3	2
5	1	5

5.3.3 矩阵的压缩存储

◻ 三元组顺序表转值算法详解

```
Status TransposeSMatrix(TSMatrix M, TSMatrix &T)
{ int p, q, col;T.mu=M.nu;T.nu=M.mu;T.tu=M.tu;
  if (T.tu) {
    q = 1;
    for (col=1; col<=M.nu; ++col)//列
      for (p=1; p<=M.tu; ++p)
        if (M.data[p].j == col) {
          T.data[q].i=M.data[p].j;
          T.data[q].j =M.data[p].i;
          T.data[q].e =M.data[p].e;  ++q;
        }
  }
  return OK;
}
```

M

1	2	1
1	5	5
2	2	7
3	1	3
3	4	2

T

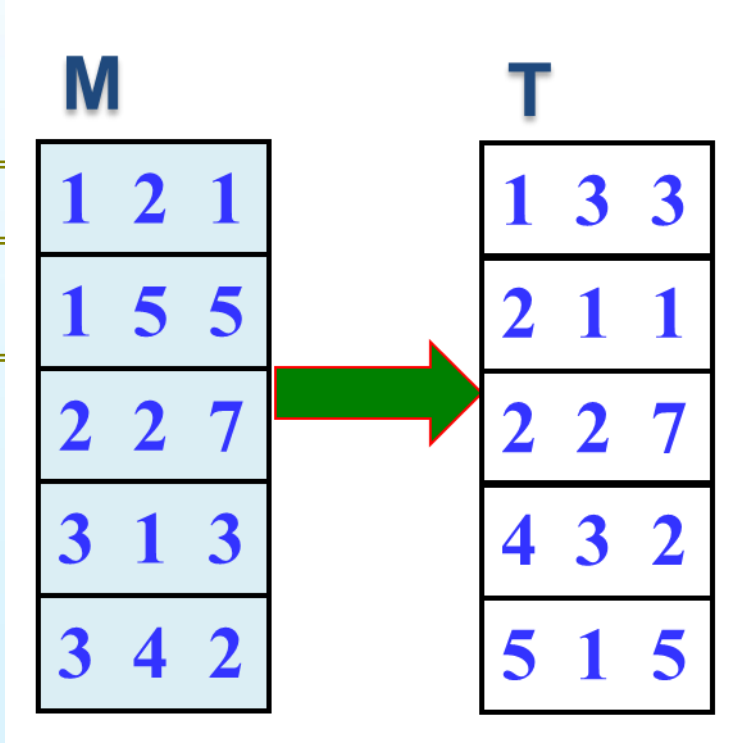
1	3	3
2	1	1
2	2	7
4	3	2
5	1	5

5.3.3 矩阵的压缩存储

- 三元组顺序表转值算法时间复杂性

```
for (col=1; col<=M.nu; ++col)
    for (p=1; p<=M.tu; ++p)
      ..............................
      }
```

时间复杂度为: O(M.nu*M.tu)



5.3.3 稀疏矩阵的压缩存储

□转置方法2

➢顺序找，跳着存

$$\begin{bmatrix} 0 & 7 & 0 & 4 & 0 \\ 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 6 & 0 & 0 & 0 \end{bmatrix}$$

矩阵M

$$\begin{bmatrix} 0 & 3 & 0 & 0 \\ 7 & 0 & 0 & 6 \\ 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

矩阵T

使用两个一维数组num和pot，

1) num[j]表示矩阵M中第j列的非零元素个数

2) pot[j]表示矩阵M中第j列的下一个非零元素在T中应存放的位置（初值为该列第一个非零元素在B中应存放的位置)。

得出： pot[1]=0

pot[j]=pot[j-1]+num[j-1]　$2 \le j \le M.n$

j	1	2	3	4	5
num[j]	1	2	0	1	1
pot[j]	0	1	3	3	4

i	j	v
1	2	7
1	4	4
2	1	3
3	5	1
4	2	6

M.data

i	j	v
1	2	3
2	1	7
2	4	6
4	1	4
5	3	1

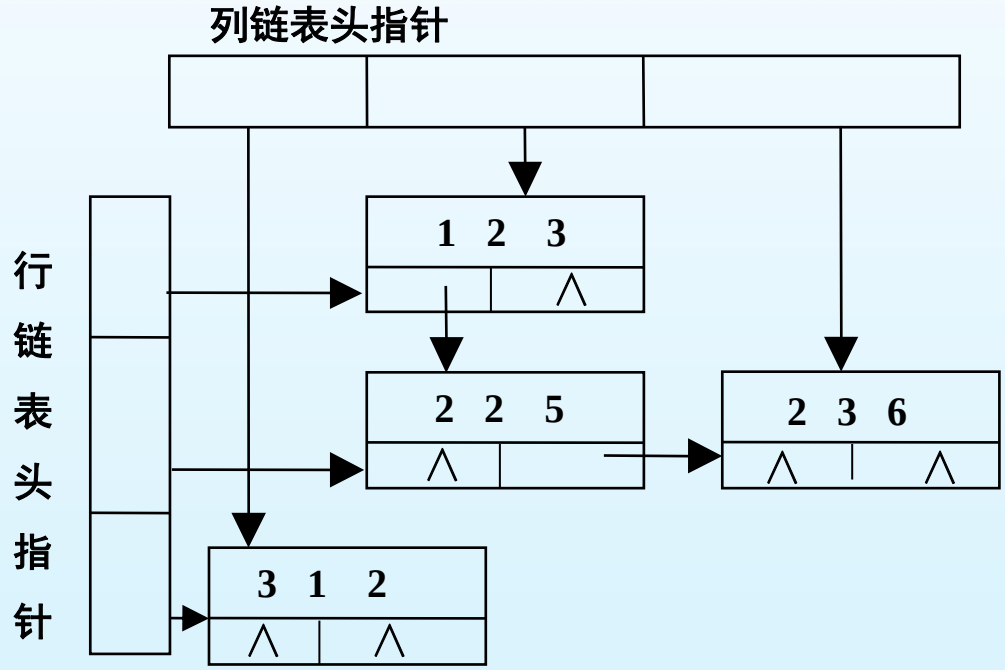
T.data

5.3.3 稀疏矩阵的压缩存储

◆用十字链表存储稀疏矩阵

- 稀疏矩阵的每个非零元素可用一个含5个域的结点表示，
- 结点结构除了表示非零元素所在的行、列和值的三元组外，还增加了两个链域：
  - ✓指向本行中下一个非零元素(同一行的后继)行指针域right
  - ✓指向本列下一个非零元素(同一列的后继)列指针域down

$$\begin{bmatrix} 0 & 3 & 0 \\ 0 & 5 & 6 \\ 2 & 0 & 0 \end{bmatrix}$$



第5章 数组和广义表

5.1 数组的类型定义

5.2 数组的顺序表示和实现

5.3 矩阵的压缩存储

5.4 广义表的类型定义

5.5 广义表的存储结构

1.2 基本概念和术语

- **数据**：是信息的载体，能够被计算机识别、存储和加工处理。如整数，实数，字符串、图象、声音等都是数据。
- **数据元素**：数据的基本单位。相当于“记录”,在计算机程序中通常作为一个整体考虑和处理。
- **数据项**: 相当于记录的“域”或字段, 是**数据不可分割的最小单位**。如学号。
- **数据对象**：性质相同的数据元素的集合。如所有班名相同的记录集合。

学号	姓名	性别	出生日期	籍贯	入学成绩	所在班级
00201	杨润生	男	82/06/01	广州	561	00计算机2
00102	石磊	男	83/12/21	汕头	512	00计算机1
00202	李梅	女	83/02/23	阳江	532	00计算机2
00301	马耀先	男	82/07/12	广州	509	00计算机3

5.4 广义表的类型定义

□ 广义表的引入

线性表要求数据元素的类型相同，在实际应用中线性表的数据类型往往不同。

例如：一个公司有董事长，总经理，秘书，人事部，分公司等等，董事长、总经理、秘书都是单个的人，而人事部、分公司又是一个组织。

如何在这种情况下应用线性表，就是广义表的范畴。

广 义 表	表长n	表深h
A=()	0	0
B=(e)	1	1
C=(a,(b,c,d))	2	2
D=(A,B,C)	3	3
E=(a,E)	2	∞
F=(())	1	2

5.4 广义表的类型定义

广义表的定义

广义表是线性表的推广，也称列表(Lists)。它是n个元素的有限序列，记作A=（$a_1,a_2,\ldots\ldots a_n$)

其中A是表名，n是广义表的长度，ai是广义表的元素，ai既可以是单个元素，也可以是广义表。

原子：如果ai是单个元素，称为原子，一般用小写字母表示;

子表：如果ai是广义表，称为子表，一般用大写字母表示。

广 义 表	表长n	表深h
A=()	0	0
B=(e)	1	1
C=(a,(b,c,d))	2	2
D=(A,B,C)	3	3
E=(a,E)	2	∞
F=(())	1	2

5.4 广义表的类型定义

广义表的术语

表头(Head)：非空广义表的第一个元素a1;
表尾(Tail)：除了表头的其余元素组成的表;
深度：广义表中括号嵌套的最大层数。

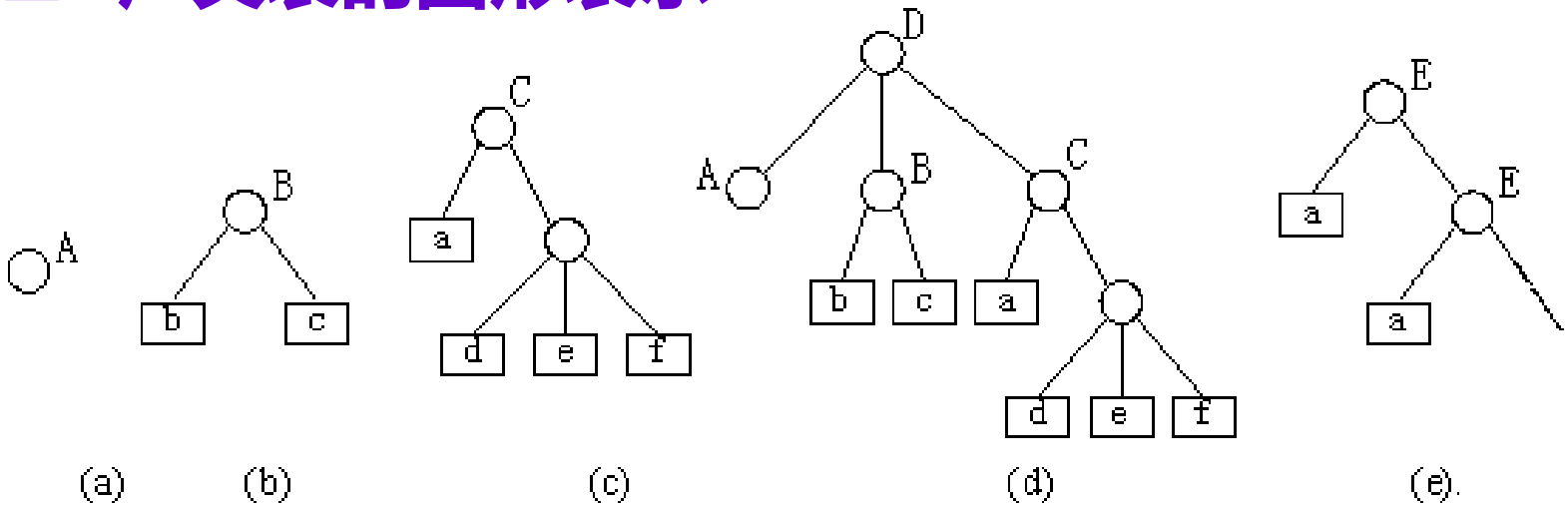
特点

广义表的元素可以是子表，子表的元素还可以是子表，存储空间难以确定，常采用链式存储。

广 义 表	表长n	表深h
A=()	0	0
B=(e)	1	1
C=(a,(b,c,d))	2	2
D=(A,B,C)	3	3
E=(a,E)	2	∞
F=(())	1	2

5.4 广义表的类型定义

□ 广义表的图形表示



广义表的图形表示

广义表的特点：

① 广义表中的元素可以是原子也可以是子表，广义表是一个多层次结构

② 广义表是可以共享的。如广义表B是D的子表

③ 广义表允许递归，如广义表E是一个递归表

④ 广义表的元素间存在次序关系和层次关系，广义表展开后包含的括号层数称为广义表的深度。如C的深度为2，E的深度为∞

5.4 广义表的类型定义

□ 基本操作举例

按例(1)的方式完成(2)(3)(4)填空

（1）B=（e）

只含一个原子，长度为1，深度为1。

（2）C=(a,(b，c，d))

有一个原子，一个子表，长度为2，深度为2。

（3）D=(B,C)　D=(B,C)=（（e）, (a,(b，c，d)))

二个元素都是列表，长度为2，深度为3。

（4）E=(a,E)

是一个递归表，长度为2，深度无限，相当于E=（a,（a,（a,（a,……）））））。

5.4 广义表的类型定义

□ 基本操作举例

例

设广义表A=(a,b,c),B=(A,(c,d)),C=(a,(B,A),(e,f)),请写出下列各运算结果：

1.GetHead(A)= a

2.GetTail(B)= ((c,d))

3.GetHead(GetHead(GetHead(GetTail(C))))= A

第5章 数组和广义表

5.1 数组的类型定义

5.2 数组的顺序表示和实现

5.3 矩阵的压缩存储

5.4 广义表的类型定义

5.5 广义表的存储结构

5.5 广义表的存储结构

◻ 广义表的头尾指针结点结构

广义表通常采用头、尾指针的链表结构

表结点：

tag=1	hp	tp

原子结点：

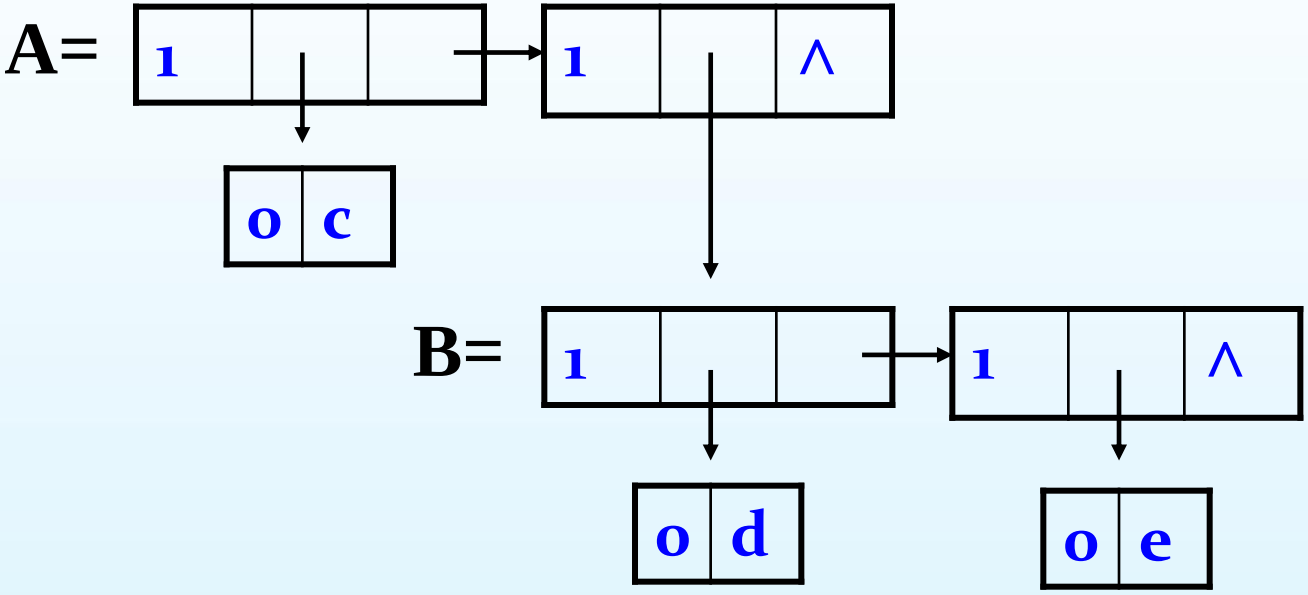
tag=0	atom

对于每一个结点，若tag=0表示这是一个原子结点，atom域存放该原子的值。若tag=1表示这是一个表结点，hp指向子表头，tp指向广义表的下一个

5.5 广义表的存储结构

广义表的头尾指针结点结构

例 画出广义表A=(c,B),B=(d,e)的存储结构图



5.5 广义表的存储结构

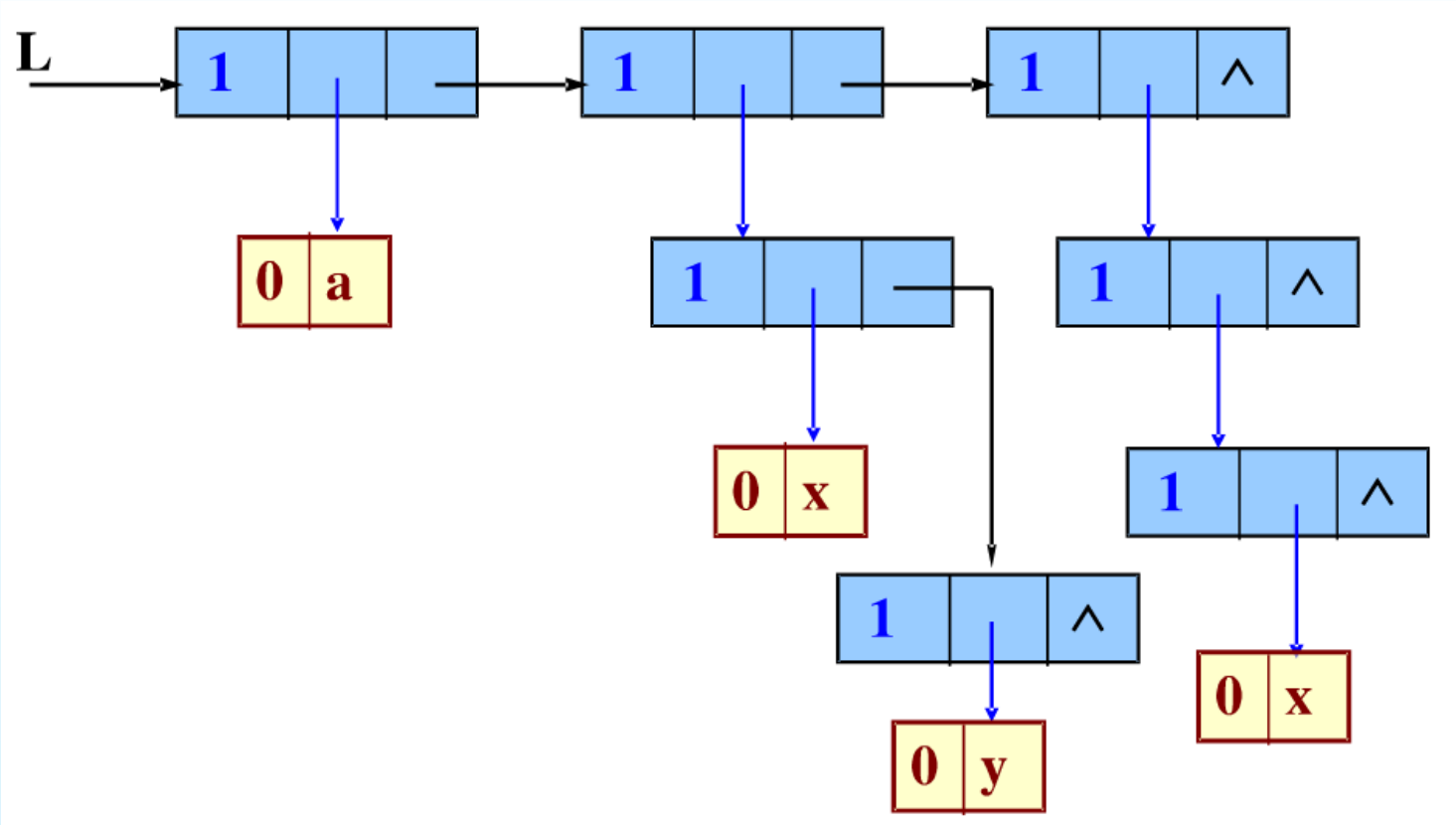
◻ 广义表的头尾链表存储表示

```
typedef enum{0,1} ElemTag;
typedef struct GLNode{
    ElemTag tag; //0,1
    union{
    AtomType atom;//原子的值
    struct {struct GLNode *hp,*tp;}ptr;
    }
}*Glist;
```

5.5 广义表的存储结构

□ 广义表的头、尾指针结点结构

例 画出广义表L=(a, (x,y), ((x)))的存储结构图



5.5 广义表的存储结构

□ 扩展头尾指针结点结构

表结点：

tag=1	hp	tp

原子结点：

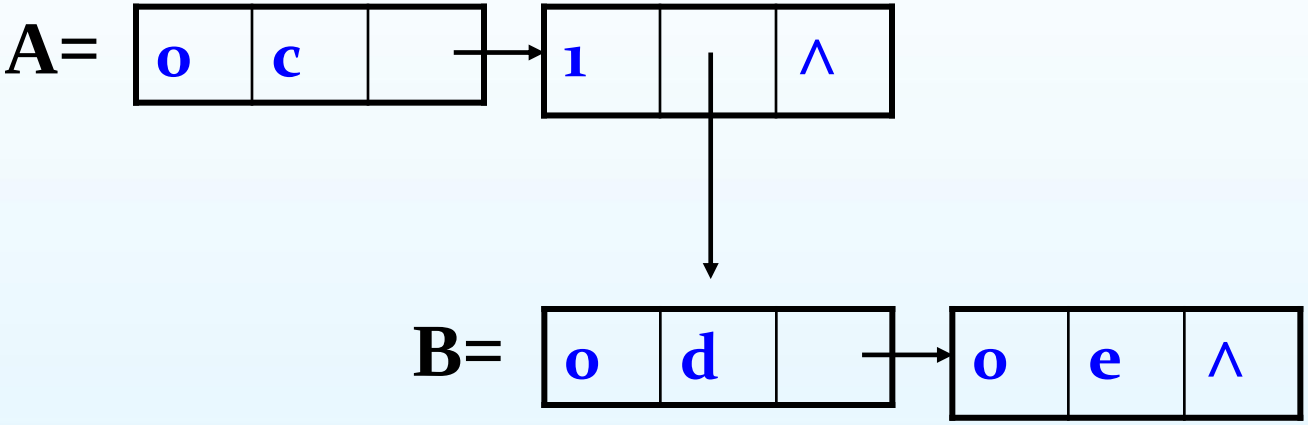
tag=o	atom	tp

对于每一个结点，若tag=0表示这是一个原子结点，atom域存放该原子的值。若tag=1表示这是一个表结点，hp指向子表头，tp指向广义表的下一个

5.5 广义表的存储结构

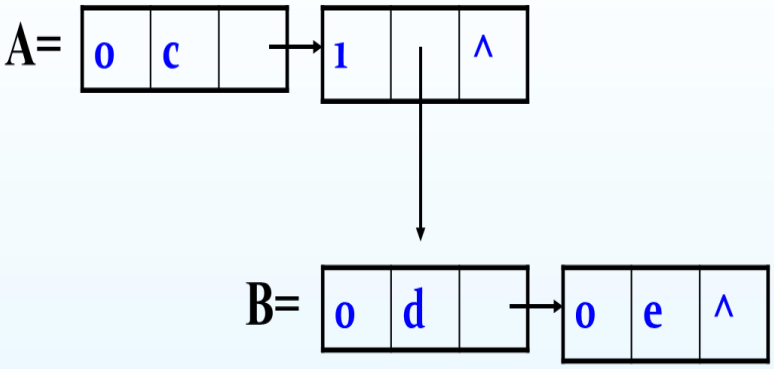
□ 扩展头尾指针结点结构

例 画出广义表A=(c,B),B=(d,e)的存储结构图



5.5 广义表的存储结构

□ 扩展头尾指针结点结构




```
typedef enum{0,1} ElemTag;

typedef struct GLNode{
    ElemTag tag;//0,1
    union{
    AtomType atom;
    struct GLNode *hp;
    };
    struct GLNode *tp;
}
```

本章小结

熟练掌握：

(1)数组的存储表示方法；

(2)数组在存储结构中的地址计算方法；

(3)特殊矩阵压缩存储时的下标变换公式；

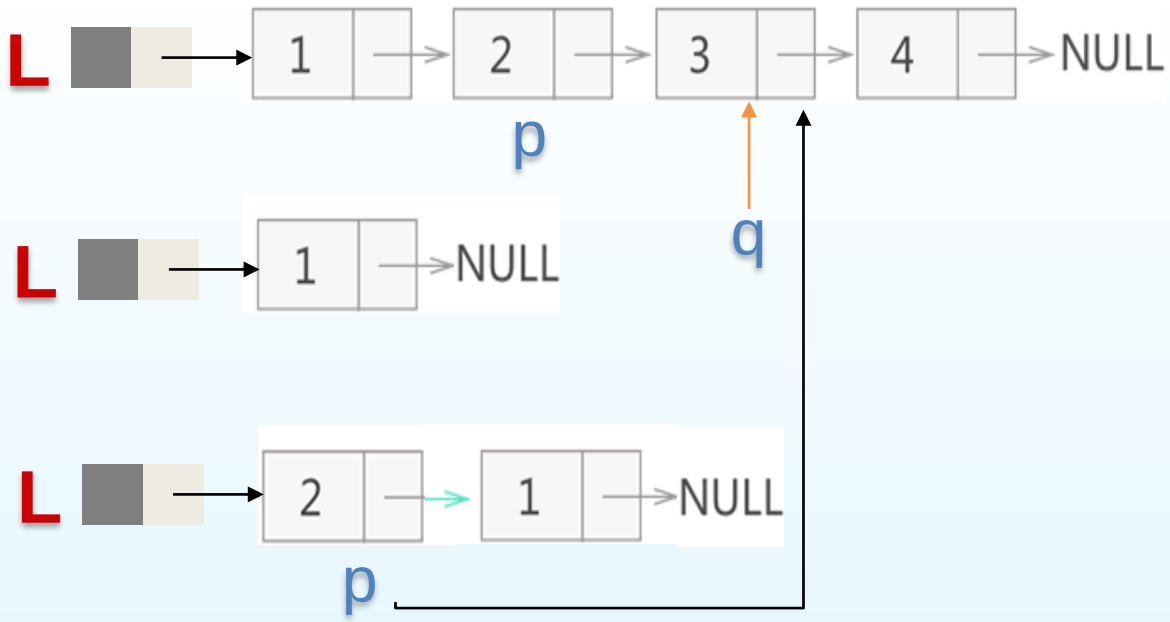
(4)稀疏矩阵的压缩存储方法；

(5)三元组表示稀疏矩阵时进行矩阵运算采用的算法。

(6)广义表的定义和性质。

例1：已知带头结点的单链表L,至少有两个元素，逆置该单链表，不许生成新结点。

```
void reverse(LIST L)
{ position p,q;
  p=L->next->next;
  L->next->next=null;
  while(p!=NULL)  {
    q=p->next;
    p->next=L->next;
    L->next=p;    p=q;
  }
}
```



例2：编写一个算法,将一个头结点指针为A的单链表A分解成两个单链表A和B。

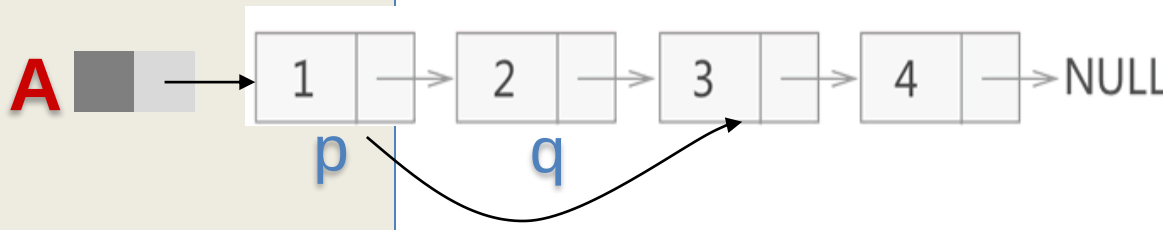
其头结点指针分别为A和B。

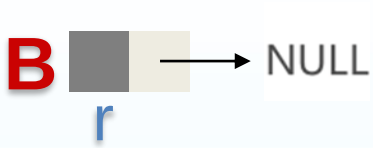
使得A链表中含有原链表A中序号为奇数的元素。

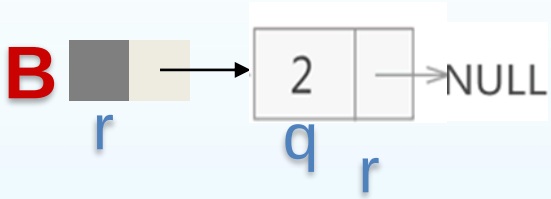
B链表中含有原链表A中序号为偶数的元素。

保持原来的相对顺序。

```
void  disa(LIST &A,LIST &B)
{LIST r,p,q;
 B->next=NULL;r=B;
 p=A->next;
while(p!=NULL&&p->next!=NULL)
{q=p->next;//q指向偶数
  p->next=q->next;//将q从原链删除
  r->next=q;r=q;//将q插入新链
  p=p->next;//p指向奇数
}
r->next=NULL;
}
```







例3：假定用一个循环单链表表示队列，该队列只设一个队尾指针rear，不设头指针，不设头结点：

写出：

(1)入队算法(入队元素为x)；

(2)出队算法。

```
void ENQUEUE(elementtype x,node *rear)
{node *p;
  p=(node *)malloc(sizeof(node));//生成新结点
  p->element=x;//x存入新结点
  if (rear==NULL)
  {rear=p;rear->next=rear;}
  else
    {p->next=rear->next;
    rear->next=p;
    rear=p;}
}
```

```
Void DEQUEUE(node *rear)
{node *p;
  if(rear==NULL)
    error(“队列空” );
  else
   {p=rear->next;
   if (rear==rear->next)
      rear=NULL;
   else
      rear->next=p->next;
   delete p;}
}
```